

# Image Processing



HW Author: Mohammad Pashaei

Instructor: Dr. Hadi Amiri

## Homework 1

You are not allowed to use any ready-made image-processing functions. You must perform all operations manually using NumPy and ... and your own mathematical implementations.

THANK YOU FOR ATTENTION TO THIS MATTER.

**Q1-Choose a colorful RGB image (3 channels) and do the following steps:**

1. Load the RGB image in Python. (Using `from PIL import Image`)
2. Convert the image into: a. a **Grayscale** image b. a **Binary (black and white)** image.
3. Display all three images: the original RGB, the Grayscale version, and the Binary version.
4. Print the array values and the **shape** of each image and compare them.

### Q2 – Manual Image Resizing (Downsampling by Averaging)

After converting the image to grayscale, perform the following steps:

1. Downsample the image to **1/2 size** by dividing the image into **2×2 blocks** and replacing each block with the **average of its pixel values**.
2. Downsample the image to **1/4 size** by dividing the image into **4×4 blocks** and replacing each block with the **average of the pixels in each block**.
3. Constraint: You are strictly forbidden from using any ready-made resize or scaling functions (such as `cv2.resize`, `PIL.resize`, etc.). The downsampling must be implemented manually using **NumPy operations or loops**.
4. Display the original grayscale image, the 1/2 scale image, and the 1/4 scale image, and compare their shapes.

### Q3-Negative of a Binary Image:

Choose a picture and make it **binary (black and white) image** where pixel values are only **0** (black) or **255** (white), perform the following:

1. Load the binary image using Pillow and convert it to a NumPy array.
2. Create the **negative** of the image **manually**, using only NumPy.
  1. Black (0) must become White (255)
  2. White (255) must become Black (0)
  3. You are not allowed to use any ready-made image-processing functions.
3. Display both the original binary image and its negative version.
4. Print and compare their array values.

---

#### Q4-Gamma Correction Application

A grayscale image is given as a NumPy array with pixel values in the range **0–255**.

- 1.Explain what **Gamma Correction** is and why it is used in image processing.
- 2.Write the mathematical formula used for Gamma Correction.
- 3.Using **NumPy only**, implement Gamma Correction for a grayscale image with a chosen gamma value (for example  $\gamma=0.5$  and  $\gamma=2.0$ ).
- 4.Display the **original image** and the **gamma-corrected images** for the two gamma values.
- 5.Briefly explain how changing the gamma value affects the brightness of the image.

---

#### Q5-Mean Filtering (3×3 Smoothing)

- 1.Explain the purpose of **smoothing (low-pass) filters** in the spatial domain.
- 2.Write the 3×3 averaging mask and explain how it operates on the image.
- 3.Implement 3×3 mean filtering **manually**:
  1. Use zero-padding or replicate-padding for borders.
  2. Use nested loops or convolution implemented by NumPy operations (but do not use any ready-made convolution functions).
- 4.Show the original and the smoothed images and discuss what happens to noise and edges.

---

#### Q6– Laplacian-Based Sharpening

- 1.Write the standard 3×3 Laplacian mask used for image sharpening.
- 2.Explain the idea of sharpening by **adding** a scaled Laplacian to the original image.
- 3.Implement the Laplacian filtering manually (no cv2.filter2D, no ready-made convolution).
- 4.Generate a sharpened version of the image and comment on the effect on edges and flat regions.

---

#### Q7 – Manual Histogram Equalization (No ready-made functions)

You are given a grayscale image and its intensity histogram  $h(r_k)$  where  $r_k$  are gray levels:

- 1.Write the mathematical formula for the **CDF-based transformation** used in histogram equalization.
- 2.Explain, in one paragraph, how histogram equalization tends to improve the contrast of an image.
- 3.Implement **manual histogram equalization** using NumPy only:
  1. Compute the normalized histogram.
  2. Compute the cumulative distribution function (CDF).
  3. Map old gray levels to new ones using the CDF.
- 4.Compare the original histogram and the equalized histogram and describe the difference.

---

## Deliverables:

For each question, your submission must include:

- The relevant **mathematical formulation**
  - A clear **conceptual explanation**
  - The **Python implementation using NumPy**
  - The **resulting output or images**
-